

Ingeniería de Software (3304)

Proyecto v 2.0

(basado en Starfighter de Jackie Wang, EECS University of York, Canadá)

Background Story

The war began many years ago, and in that time, much has changed. Thanks to dedicated research, today's Starfighters are modular, with customizable parts for weaponry, defense, and propulsion to suit any combat role. The recovery of enemy components—orbments and foci—has even unlocked unique special abilities. Our understanding of the enemy has also deepened through relentless engagement, leading to the classification of five unit types: the basic Grunt, its superior Fighter variant, the large Carrier that swarms the battlefield with Interceptor drones (which act as living shields), and the massive, support-class Pylon that heals its allies.

These enemies now demonstrate adaptive behaviors, responding to the Starfighter's presence and tactics. Consequently, the simulation from the previous Starfighter must be updated to reflect these new technologies and intelligence.

Game Setup and Core Concepts

The software, once launched, has a number of states for its progression. The logical sequence is you begin in a *not-started* state, move to the *setup* stage (which itself consists of five states) before moving into an *in-game* state. In the following, we describe each of these states.

1. not-started state: you arrive in this state as soon as you launch the software.
2. setup stage: Invoking the *play* command moves the player from the *not-started* state to the beginning of the setup stage. The setup stage consists of five states, weapon setup, armour setup, engine setup, power setup, setup summary.
 - a. Navigating between each state in the setup stage is done via the *setupBack* and *setupNext* commands. Each of these commands has an integer parameter between 1 and 5. Using the *setupBack* or *setupNext* command moves the state from the current state, respectively, backward or forward by the specified integer amount of states.

Consequently, going before the weapon setup state (e.g., invoke *setupBack*(2) when the current state is armour setup, the 2nd setup state) will lead you to back to the not-started state. Similarly, going beyond the setup summary state (e.g., invoke *setupNext*(4) when the current state is armour setup) will lead you to the in-game state.

For example, say after invoking the *play* command initially, we invoke the following in sequence: *setupNext*(2), *setupBack*(1), and *setupNext*(5). What will be the intermediate states?

- i. We start at weapon setup due to the play command.
- ii. By invoking `setupNext(2)`, we move to the engine setup state, which is two states ahead of weapon setup.
- iii. By invoking `setupBack(1)`, we move back one state which is armour setup.
- iv. By invoking `setupNext(5)`, given that setup summary is 3 states ahead of the current state (armour setup), we go beyond the setup summary state and arrive in the in-game state.

It is also possible to invoke the abort command to exit from the setup stage and go back to the not-started state.

- b. Except for the setup summary state, each of the other four states of the setup stage allows the player to choose the equipment (i.e., weapon, armour, engine, and power) for the Starfighter. By default, option 1 is selected for the weapon, armour, engine, and power. By invoking the setup select command. The setup select command takes as input an integer between 1 and 5 and selects the corresponding option displayed for that particular state in the setup stage. Selected equipment is preserved; you can overwrite the selected equipment by invoking the setup select command; the history of selected equipment is wiped out only when the software is terminated. This means that if a certain equipment is chosen and the setup stage is exited from, the next time the setup stage is entered, that certain equipment will be selected by default.

The setup summary summarizes what equipment the Starfighter is entering the game with.

There are 5 different types of weapons, 4 different types of armour, 3 different types of engines, and 5 different types of power. Weapons, armour, and engines share some common attributes (see below). Attribute values for each weapon, armour, and engine, along with the errors corresponding to selecting an invalid equipment option, can be found later on in this document.

The Starfighter's final attribute values, upon entering the in game state, are the sum of its parts. Attributes which weapon, armour, and engine share in common include:

- i. Health: It represents the maximum integrity of the entity (e.g., Starfighter, Grunt), also known as the total health. An entity's current health value has the lower bound of 0 and upper bound of this maximum Health value. If the current health falls to 0, the entity is destroyed. Note that in the game, for the Starfighter only, it is possible for its current health value to exceed this maximum Health value by using a power.
- ii. Energy: It represents the maximum energy of the Starfighter, also known as the total energy. Note that the current energy cannot exceed total energy (except when using a power). Energy is required for the Starfighter to move, to use a power, or to fire projectiles.
- iii. Regen (for regeneration): There are two numbers.
 1. The first number represents the amount of current health the Starfighter gains per turn passively (health regeneration). Recall

that the current health cannot exceed total health, so regeneration only works up to the total health value (except with the use of a power).

2. The second number is for energy regeneration and it works the same way as health (with respect to the Energy value above). Note. Regen is the base value for the (health or energy) regeneration process to take place.
 - iv. Armour: If the Starfighter and a projectile collide, the damage done to the Starfighter is the projectile's damage minus the Starfighter's armour. Armour does not have any cancelling effect on collisions with non-projectiles.
 - v. Vision: It represents how many spaces away, on the board, the Starfighter can see from its current location. For example, if the Starfighter is in [3,0], to see what entity is in [2,2], you need a vision of 3 (1 vertical + 2 horizontal) or more. Spaces beyond the range of the Starfighter's vision are known to be in the fog of war. In the board, they are represented by a ? symbol.
 - vi. Move: It represents how many spaces away the Starfighter can move from its current location (assuming it has enough energy). The distance calculation is the same as vision calculation: vertical distance plus horizontal distance in terms of spaces.
 - vii. Move Cost: It represents how many energy units it takes for the Starfighter to move 1 space. If the move cost is 5 and we make a move from [0,0] to [3,2], it would cost 25 energy units: 5 spaces (3 vertical + 2 horizontal) $\times$ 5.
3. in-game state: this is when the game has begun. See later sections.

Weapon Types

The weapon chosen plays a role in how the Starfighter's fire command affects the board when used in game. Each weapon has the shared attributes described above (e.g., health, energy). In addition, each weapon has other attributes that determine: 1) the health or energy cost to fire the projectiles; and 2) the base damage of the projectiles (when they collide with other entities). These attributes can also be found later on in this document. Here we describe:

- When the Starfighter fires, how many projectiles spawn and where they spawn.
- After spawning, assuming the friendly projectiles are still on the board, every subsequent turn later, how the friendly projectiles act also depends on the weapon selected.

There are 5 different types of weapon:

- Standard: One projectile is spawned. It appears to the right of the Starfighter. For example, if the Starfighter is at [0,1], the projectile appears at [0,2]. Every subsequent turn after, the projectile moves 5 units to the right.

- Spread: Three projectiles are spawned:
 - The first one is spawned to the top-right of the Starfighter, the second to the right of the Starfighter while the last projectile is spawned to the bottom-right of the Starfighter. For example:
 - If the Starfighter is at [3,3], the projectiles appear at [2,4], [3,4] and [4,4] in that order.
 - Every subsequent turn after, the projectiles move 1 unit in the direction where they spawned relative to the Starfighter when it fired. For example, the projectile at [2,4] will move up right to [1,5] while the projectile at [3,4] will move to [3,5].

Note. Recall how the Starfighter travels diagonally: vertical first, then horizontal. Contrast this with how a projectile travels diagonally (which is a real diagonal movement). For example, if there was an entity at [1,4], no collision will occur from the projectile travelling from [2,4] to [1,5]. On the other hand, if the Starfighter were to travel from [2,4] to [1,5], it would first travel vertically to [1,4], resulting in a collision.
- Snipe: One projectile is spawned. It appears to the right of the Starfighter. For example, if the Starfighter is at [0,1], the projectile appears at [0,2]. Every subsequent turn after, the projectile moves 8 units to the right.
Unlike all the other projectiles, this projectile travels by “teleporting” to the final location. More precisely, if there are any entities along the horizontal path to the targeted location, the projectile will not collide with any of those entities unless there is an entity in the targeted location (which is 8 units to the right of the initial location).
- Rocket: Two projectiles are spawned. The first one is spawned to the top-left of the Starfighter while the second projectile is spawned to the bottom-left of the Starfighter. For example, if the Starfighter is at [1,1], the projectiles appear at [0,0] and [2,0] in that order. Every subsequent turn after, the projectiles move starting at 1 unit to the right and doubling in movement each turn.
If the Starfighter is in the first column and they fire, the projectiles will be spawned outside the board behind them, in which case we still assign ids to them. However, in the next turn, since these projectiles are already outside the board, we do not keep track of them, meaning that they will not move into the board.
- Splitter: One projectile is spawned. It appears to the right of the Starfighter. For example, if the Starfighter is at [0,1], the projectile appears at [0,2]. The projectile does not move.

Specials

There are 5 different kinds of special/power a Starfighter may have. Powers may be used in game via the Starfighter’s special command. The cost, type of cost, and a brief description of each kind of power can be found later on in this document. Below is a more detailed description of each power.

- Recall: When you start a game, the Starfighter spawns in a certain location. When this special is used, teleport the Starfighter to that location. Unlike moving, you can still use

this command if the Starfighter is to be teleported to the same location. Make sure to account for collisions at the spawn location.

- Repair: Increase current health by 50 units. This is the only case that current health may exceed the total health for a Starfighter.
- Overcharge: This is to sacrifice current health for gaining current energy. This is dependent on the amount of the current health spent. It will use up to 50 current health as long as the Starfighter is not destroyed to convert to current energy. The conversion is: 1 current health for 2 current energy.
For example: If you have 1 current health, 0 current health will be converted to 0 current energy. If you have 11 current health, 10 current health will be converted to 20 current energy, and there is still 1 current health left. If you have 100 current health, 50 current health will be converted to 100 current energy, and there is still 50 current health left. This is the only case that current energy may exceed total energy for a Starfighter.
- Deploy Drones: Clears out all the projectiles on the board. The order of removal is from projectiles that spawned earliest to latest.
- Orbital Strike: All enemies take 100 damage, reduced by their armour value. Suppose an enemy has 15 armour, they will take 85 (i.e., $100 - 15$) damage. The order of taking damage is from enemies that spawned earliest to latest.

Enemies

There are various kinds of enemies, and they have shared attributes, including:

- health (total health)
- regen (health regeneration per turn)
- armour (which mitigates damage: one point of damage to one point of armour for the collision with a friendly projectile or an Orbital Strike by the Starfighter)
- vision (which works the same way as the Starfighter's vision)
- seen by Starfighter (whether or not the enemy is within the Starfighter's vision; only updated in certain phases of a turn; see Phase 4 and Phase 6)
- can see Starfighter (whether or not the Starfighter is within the enemy's vision; only updated in certain phases of a turn; see Phase 4 and Phase 6)

Enemies may also preempt certain moves depending on the Starfighter's actions, and act differently depending on whether they see the Starfighter or not. Furthermore, enemies spawn other entities, and spawned entities do not act on the turn they are spawned. Entities that spawn outside the board must have an id associated with them, but there is no need to keep track of them. Here are the kinds of enemies with their initial attribute values and how they act:

1. Name: Grunt, Health:100, Regen:1, Armour:1, Vision:5.
 - Preemptive Action: If the Starfighter passes, increase both current health and total health by 10. If the Starfighter uses a special, increase current health and total health by 20. Turn does not end in either case.

- Action when Starfighter is not seen: Moves 2 spaces left. If the Grunt is still on the board and is not destroyed, fire a projectile which spawns directly to the left of the Grunt. The spawned projectile moves left 4 spaces per turn and has a base damage of 15.
 - Action when Starfighter is seen: Moves 4 spaces left. If the Grunt is still on the board and is not destroyed, fire a projectile which spawns directly to the left of the Grunt. The spawned projectile moves left 4 spaces per turn and has a base damage of 15.
2. Name: Fighter, Health:150, Regen:5, Armour:10, Vision:10.
- Preemptive Action: If the Starfighter fires, increase armour by 1 and do not end the turn. If the Starfighter passes, move 6 spaces left. If the Fighter is still on the board and is not destroyed, fire a projectile which spawns directly to the left of the Fighter. The spawned projectile moves left 10 spaces per turn and has a base damage of 100. End the Fighter's turn.
 - Action when Starfighter is not seen: Moves 3 spaces left. If the Fighter is still on the board and is not destroyed, fire a projectile which spawns directly to the left of the Fighter. The spawned projectile moves left 3 spaces per turn and has a base damage of 20.
 - Action when Starfighter is seen: Moves 1 space left. If the Fighter is still on the board and is not destroyed, fire a projectile which spawns directly to the left of the Fighter. The spawned projectile moves left 6 spaces per turn and has a base damage of 50.
3. Name: Carrier, Health:200, Regen:10, Armour:15, Vision:15.
- Preemptive Action: If the Starfighter uses a special, increase regen by 10 and do not end the turn. If the Starfighter passes, move 2 spaces left. If the Carrier is still on the board and is not destroyed, spawn two Interceptors: one directly above it and then one directly below it. End the Carrier's turn.
Note. If an enemy is occupying a spawn location for the Interceptor, do not spawn the Interceptor. If the spawn location is outside the board, treat it similar to a projectile spawning outside of the board. This means spawn the Interceptor (including assigning an ID to it, which will be discussed later), but do not keep track of that entity as it is out of the board.
 - Action when Starfighter is not seen: Moves 2 spaces left.
 - Action when Starfighter is seen: Moves 1 space left. If the Carrier is still on the board and is not destroyed, spawn an Interceptor directly left of the Carrier.
4. Name: Interceptor, Health:50, Regen:0, Armour:0, Vision:5.
- Preemptive Action: If the Starfighter fires, attempt to move the Interceptor vertically directly to the row the Starfighter is in before ending the Interceptor's turn. More precisely about the vertical movement:
 - i. Case 1. If the Interceptor and the Starfighter are in different columns, then:
 - If the path to the row where the Starfighter is in is clear, then move to that row (without colliding with the Starfighter).

- If there is at least one occupying enemy blocking the path, move as close to the first one (so as to avoid a collision). Along the way to this first occupying enemy, the Interceptor may collide with projectiles.
 - If there is no occupying enemy blocking the path, but there is one or more occupying projectiles, then the Interceptor collides with those projectiles.
 - ii. Case 2. If the Interceptor and the Starfighter are in the same column, then:
 - If the path to the Starfighter's location is clear, then collide with the Starfighter.
 - If there is at least one occupying enemy blocking the path, move as close to the first one (so as to avoid a collision). Along the way to this first occupying enemy, the Interceptor may collide with projectiles.
 - If there is no occupying enemy blocking the path, but there is one or more occupying projectiles, then the Interceptor collides with those projectiles before it may collide with the Starfighter.
 - iii. Note for Case 1 and Case 2. The Interceptor may be destroyed along the path, due to collisions with projectiles, in which case it is just removed from the board without continuing moving along the path.
- Action when Starfighter is not seen: Moves 3 spaces left.
- Action when Starfighter is seen: Moves 3 spaces left.
- 5. Name:Pylon, Health:300, Regen:0, Armour:0, Vision:5.
 - Preemptive Action: None.
 - Action when Starfighter is not seen: Moves 2 spaces left. If the Pylon is still on the board and is not destroyed, then for each enemy within the Pylon's vision range (including itself), heal it for 10 current health. Note that current health cannot exceed total health when healing this way.
 - Action when Starfighter is seen: Moves 1 space left. If the Pylon is still on the board and is not destroyed, fire a projectile which spawns directly to the left of the Pylon. The spawned projectile moves left 2 spaces per turn and has a base damage of 70.

Initial Game State

The initial board output when the game is started is displayed below:

```

- - - - - - - - - - - ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
- - - - - - - - - - - ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
- - - - - - - - - - - ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
- - - - - - - - - - - ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
S - - - - - - - - - - - ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
- - - - - - - - - - - ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
- - - - - - - - - - - ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
- - - - - - - - - - - ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
- - - - - - - - - - - ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
- - - - - - - - - - - ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
- - - - - - - - - - - ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?

```

In this example:

- There are 10 rows and 30 columns, meaning that the play command used must have been invoked passing as arguments 10 rows and 30 columns.
- The Starfighter's vision is 13 as any location beyond 13 spaces away (sum of vertical and horizontal displacement from [4,0]) is covered by the fog of war (which is represented by ?), denoting the fact the entity there is unknown.
- The Starfighter does not always spawn at [4,0]. Spawn is dependent on the number of rows specified for the board. Similar to the first version of the project, the Starfighter will always spawn in column 1. The row it spawns in is the ceiling of the number of rows divided by 2. The Starfighter's attributes are the sum of the parts chosen for it.

Symbols

Below is the list of symbols that may appear on the board.

- ? : It represents the fog of war, i.e., from the perspective of the Starfighter, locations that are too far away for the Starfighter to see what entity is in that location.
- _ : This symbol means the location is not occupied by any entity.
- G : This symbol means the location is occupied by a Grunt.
- F : This symbol means the location is occupied by a Fighter.
- C : This symbol means the location is occupied by a Carrier.
- I : This symbol means the location is occupied by an Interceptor.
- P : This symbol means the location is occupied by a Pylon.
- < : This symbol means the location is occupied by an enemy projectile (fired by some kinds of enemies).
- S : This symbol means the location is occupied by the Starfighter (which fires friendly projectiles).
- * : This symbol means the location is occupied by a friendly projectile.
- X : This symbol means the location is where the Starfighter was destroyed. This symbol is only present when the game is over due to destruction of the Starfighter.

Entity IDs

Each entity has a unique integer ID associated with it.

- The Starfighter has an ID of 0.
- Projectiles have an ID starting at -1, decreasing by 1 for each new (enemy or friendly) projectile spawned.
- Enemies have an ID starting at 1, increasing by 1 for each new enemy spawned.
- IDs are reset between games so the counters for IDs start at 1 and -1 respectively for the next game.

A Turn of the Game

A turn occurs when the move, pass, fire or special command is invoked successfully (with no errors) when in game. Later on we describe the situations where such commands are blocked, and thus their execution would lead to an error.

A turn comprises multiple phases.

Phase 1: Friendly Projectiles Act

Friendly projectiles (fired by the Starfighter) on the board move one by one, in the order of oldest projectile to newest (in terms of when the projectile was spawned).

Movement of each friendly projectile is determined by the weapon of the Starfighter. For the Snipe weapon, projectiles teleport directly to the destination, rather than travelling there.

Possible interactions between friendly projectiles and other entities include:

- Colliding with an enemy projectile: There are three cases to consider. First, if the enemy projectile's damage is higher, the friendly projectile is removed from the board. The new enemy projectile's damage is its damage subtracted by the friendly projectile's damage. Second, if the friendly projectile's damage is higher, the enemy projectile is removed from the board. The new friendly projectile's damage is its damage subtracted by the enemy projectile's damage. The friendly projectile continues its travel to its end destination. Third, if damages of both projectiles are the same, they are both removed from the board.
- Colliding with a friendly projectile: The stationary projectile is removed from the board. The moving friendly projectile's damage gets added with the damage of the stationary projectile. The moving friendly projectile continues its travel to its end destination.
- Colliding with an enemy: The friendly projectile is removed from the board. Say the friendly projectile has damage d , and the enemy has current health h and armour value a . The enemy's current health is updated to: $h - \max(d - a, 0)$. That is, when $a \geq d$, there is no harm to the current health. If the enemy's current health becomes either 0 or below, then after the collision, the enemy is destroyed and removed from the board.
- Colliding with the Starfighter: The friendly projectile is removed from the board. Say the friendly projectile has damage d , and the Starfighter has current health h and armour value a . The Starfighter's current health is updated to: $h - \max(d - a, 0)$. That is, when $a \geq d$, there is no harm to the current health. If the Starfighter's current health becomes 0

or below after the collision, the Starfighter is destroyed and the game is over. In this case, set the Starfighter's current health to 0, skip newer friendly projectiles' actions, and skip the rest of the turn. Change the Starfighter's symbol on the board to an X and output the state when the Starfighter got destroyed.

Phase 2: Enemy Projectiles Act

Enemy projectiles (fired by certain kinds of enemies) on the board move one by one, in the order of oldest projectile to newest (in terms of when the projectile was spawned).

Movement of each enemy projectile is determined by the kind of enemy which fired it.

All enemy projectiles travel to their end destination rather than teleporting there directly.

Possible interactions between enemy projectiles and other entities include:

- Colliding with a friendly projectile: There are three cases to consider. First, if the friendly projectile's damage is higher, the enemy projectile is removed from the board. The new friendly projectile's damage is its damage subtracted by the enemy projectile's damage. Second, if the enemy projectile's damage is higher, the friendly projectile is removed from the board. The new enemy projectile's damage is its damage subtracted by the friendly projectile's damage. The enemy projectile continues its travel to its end destination. Third, if damages of both projectiles are the same, they are both removed from the board.
- Colliding with an enemy projectile: The stationary projectile is removed from the board. The moving enemy projectile's damage gets added with the damage of the stationary projectile. The moving enemy projectile continues its travel to its end destination.
- Colliding with an enemy: The enemy projectile is removed from the board. The enemy is healed: its current health is increased by the enemy projectile's damage value. Note that armour does not affect the amount healed. If the enemy's current health is higher than its total health after the heal, set its current health to the total health.
- Colliding with the Starfighter: The enemy projectile is removed from the board. Say the enemy projectile has damage d , and the Starfighter has current health h and armour value a . The Starfighter's current health is updated to: $h - \max(d - a, 0)$. That is, when $a \geq d$, there is no harm to the current health. If the Starfighter's current health becomes 0 or below after the collision, the Starfighter is destroyed and the game is over. In this case, set the Starfighter's current health to 0, skip newer enemy projectiles' actions, and skip the rest of the turn. Change the Starfighter's symbol on the board to an X and output the state when the Starfighter got destroyed.

Phase 3: Starfighter Act

First, let us define the notion of regeneration. The Starfighter has a regen attribute denoted by two numbers (e.g., 1/2): when regeneration occurs, there is health and energy regeneration. For health regeneration, if the current health of the Starfighter is less than the total health, add the first number of the regen attribute to the current health. If current health exceeds total health

after the previous step, set current health to total health. If initially, the current health exceeds total health via special usage, do not apply health regeneration. Energy regeneration works the same way, except that it is denoted by the second number of the regen attribute. The Starfighter acts according to the command being invoked by the user:

- pass: Apply regeneration twice. The Starfighter does not do anything.
- move: Apply regeneration. Subtract the cost of moving from energy. The energy cost is the move cost multiplied by the amount of spaces between the current location and the end location (e.g., to move from [0,1] to [1,5], there are $1 + 4 = 5$ spaces). The “end location” does not necessarily mean the specified location, but where the Starfighter stops. The Starfighter may be destroyed on its way to the specified location. Similar to the first version of the project, the Starfighter travels vertically then horizontally to the targeted location, interacting with other entities on the way. If the Starfighter’s current health is 0 at any point, the Starfighter is destroyed and the game is over, meaning that the Starfighter does not continue to move to the targeted destination, and the remaining phases for the turn are skipped. Change the Starfighter’s symbol on the board to an X and output the state when the Starfighter got destroyed.

Possible interactions between the Starfighter and other entities include:

- Colliding with an enemy or a friendly projectile. The projectile is removed from the board. Say the projectile has damage d , and the Starfighter has current health h and armour value a . The Starfighter’s current health is updated to: $h - \max(d - a, 0)$.
- Colliding with an enemy. The enemy is destroyed and removed from the board. Say the enemy has current health eh , and the Starfighter has current health sh . The Starfighter’s current health is updated to: $sh - eh$ (i.e., armour plays no role).

For both of the above cases of collision, if the Starfighter’s current health becomes 0 or below after the collision, the Starfighter is destroyed and the game is over. In this case, set the Starfighter’s current health to 0 and skip the rest of the turn. Change the Starfighter’s symbol on the board to an X and output the state when the Starfighter got destroyed.

- fire: Apply regeneration. Subtract the cost of firing, and spawn (friendly) projectiles based on the weapon the Starfighter is equipped with. The order of the projectile spawn is determined by the row they spawn in: spawning the projectiles closer to row A first (equivalently, spawning the projectiles from top to bottom of the board).

Possible interactions between the friendly projectiles spawned and other entities include:

- Spawning in a location that has an enemy projectile. There are three cases to consider. First, if the enemy projectile’s damage is higher, the friendly projectile is removed from the board. The new enemy projectile’s damage is its damage subtracted by the friendly projectile’s damage. Second, if the friendly projectile’s damage is higher, the enemy projectile is removed from the board. The new friendly projectile’s damage is its damage subtracted by the enemy projectile’s damage. Third, if damages of both projectiles are the same, they are both removed from the board. For the first and third cases, even though the friendly

projectile is removed from the board after spawning, an ID needs to be associated with it.

- Spawning in a location that has a friendly projectile. The projectile already in that location is removed from the board. The new spawning friendly projectile's damage is added by the damage of the removed projectile.
- Spawning in a location that has an enemy. The friendly projectile is removed from the board. Say the friendly projectile has damage d , and the enemy has current health h and armour value a . The enemy's current health is updated to: $h - \max(d - a, 0)$. That is, when $a \geq d$, there is no harm to the current health. If the enemy's current health becomes either 0 or below, then after the collision, the enemy is destroyed and removed from the board. Even though the friendly projectile is removed from the board after spawning, an ID needs to be associated with it.
- Though not an interaction, if the friendly projectile spawns outside of the board, you don't have to keep track of it, but it needs an ID for outputting messages.
- special: Apply regeneration. Subtract the cost for using the special. What happens depends on the special the Starfighter has.

Phase 4: Enemy Vision Update

Update the seen by Starfighter and can see Starfighter attributes for each enemy that is still on the board:

- For seen by Starfighter, set to true if the sum of the horizontal and vertical displacement between the enemy's location and the Starfighter's location is less than or equal to the Starfighter's vision. Set to false otherwise.
- For can see Starfighter, set to true if the sum of the horizontal and vertical displacement between the enemy's location and the Starfighter's location is less than or equal to the enemy's vision. Set to false otherwise.

Phase 5: Enemies Act

Depending on the Starfighter's action, certain enemies may preempt their actions.

- All the preempted actions for the enemies on the board go first. The order is from oldest to newest spawned enemies on the board.
- There are two possible consequences of each preempted action. First, a preempted action may not end the enemy's turn, meaning it can act again later. Second, a preempted action may cause the enemy's turn to end, which works the same way as a normal non-preempted action does (see the next point).

After the preempted actions, all enemies on the board whose turn did not end will act from the order from oldest to newest spawned on the board.

How an enemy acts depends on whether they see the Starfighter or not and the type of the enemy they are. An enemy's action can be generalized as the following sequence of actions:

- Regeneration, which works the same way as the Starfighter's regeneration. The only difference being is that there is no energy regeneration for an enemy.
- Movement. The enemy will try to move to the targeted location.
 - If the path they are going through contains another enemy, the enemy will stop 1 space before hitting that enemy, while still interacting with other entities the same way as enemy interactions when spawning on the way there (see below).
 - Otherwise, travel through the path, interacting with other entities the same way as enemy interactions when spawning, stopping only when the enemy is destroyed (and removed from the board) or is at the targeted location. There is no penalty if the enemy escapes (move out of the board), and such an escaping enemy is not considered to be destroyed for scoring purposes.
- If the enemy is still on the board, they may either fire or spawn enemies. Recall that when an enemy fires, it spawns enemy projectiles.
 - Enemy projectile spawning interacts with the other entities the same way with enemy spawning (described later on), except the targeted location is where they spawn.
 - Enemy spawning interacts with the other entities as described below (enemy spawning). An ID still needs to be assigned if the spawn location is outside the board. Similar to enemies naturally spawning, an enemy will not spawn in a location occupied by another enemy. Spawned enemies do not act the turn they are spawned.

If the Starfighter's current health becomes 0 at any point, the Starfighter is destroyed and the game is over. In this case, set the Starfighter's current health to 0, finish the current enemy's action, skip other newer enemies' actions, and skip the rest of the turn. Change the Starfighter's symbol on the board to an X and output the state when the Starfighter got destroyed.

Phase 6: Enemy Vision Update

Same as Phase 4. Note that these attributes ARE NOT modified at all in Phase 5 after each enemy acts for simplicity.

Phase 7: Enemy Spawn

Enemies are spawned randomly. Although you are free to decide how random enemy spawning is implemented, a potential implementation may be based on the following steps:

- Randomly decide if an enemy is going to be spawned or not.
- If an enemy is to be spawned, randomly generate a position in the board where to spawn the enemy.
- If an enemy is to be spawned, randomly decide what type of enemy is going to be spawned.

You may implement your own mechanisms to make some enemy types more likely than others, and make it part of your game setup stage.

Possible interactions between the enemy spawned and other entities include:

- Spawning in a location occupied by a friendly projectile. The friendly projectile is removed from the board. Say the friendly projectile has damage d , and the enemy has current health h and armour value a . The enemy's current health is updated to: $h - \max(d - a, 0)$. That is, when $a \geq d$, there is no harm to the current health. If the enemy's current health becomes either 0 or below, then after the collision, the enemy is destroyed and removed from the board.
- Spawning in a location occupied by an enemy projectile. The enemy projectile is removed from the board. The enemy is healed: its current health is increased by the enemy projectile's damage value. Note that armour does not affect the amount healed. If the enemy's current health is higher than its total health after the heal, set its current health to the total health.
- Spawning in a location occupied by the Starfighter. The enemy is destroyed and removed from the board. Say the enemy has current health eh , and the Starfighter has current health sh . The Starfighter's current health is updated to: $sh - eh$ (i.e., armour plays no role). If the Starfighter's current health becomes 0 or below after the collision, the Starfighter is destroyed and the game is over. In this case, set the Starfighter's current health to 0. Change the Starfighter's symbol on the board to an X and output the state when the Starfighter got destroyed.

Game Over

When in-game, the way a game can be over is if the user uses the abort command. Another way is if the Starfighter's current health is reduced to 0 and thus destroyed.

Game Won

You cannot win a game. A user can play indefinitely as long as the game is not over.

Tasks

Starting with the group's solution to the previous part of this project, develop a fully functional Starfighter game with command line interface, according to the problem description above. For the implementation:

- a. Use BDD+TDD. Use input space partition approaches to derive BDD tests. Employ at least pairwise and base block coverage criteria (at least once each), and explain the rationale behind selecting each one. Tests must achieve high (over 80%) instruction and branch coverage.
- b. Apply all the learned principles on clean code to your development.
- c. Choose at least three of the design patterns we have learned in the course so far, and apply them to your project.
- d. Provide a UML diagram explaining your design. Describe and justify the design patterns that you applied.
- e. Promote balanced and consistent contribution across the entire team in the code repository.
- f. Code sharing between different groups is strictly prohibited.

Deadline: 16/11, 23.59hs.

Oral presentation: To be defined.